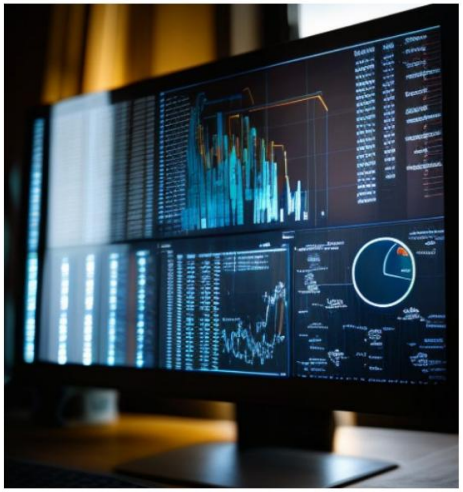
Machine Translated by Google

[Chap 3] Fundamentals of Data Processing and Numerical Operations



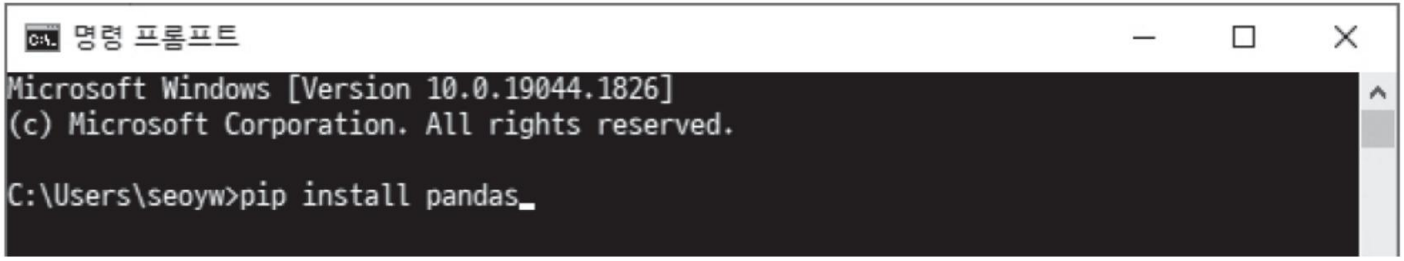
Machine Translated by Google

Install Pandas

- Pandas package
  - Functions such as reading data from a file, performing various processing, and writing to a file – As an external package, separate installation is required
    - Installation command: pip install OOO
- Installing Pandas –

  Run cmd (Command Prompt) and type 'pip install pandas' •

  On macOS, run Terminal and type 'pip3 install pandas'



- Importing external packages to use in a Python program

문법 import … as문

import(패키지명) as (별명)

Machine Translated by Google

Pandas Basic Examples

- Prepare a data file in Excel (.csv) format

scores.csv - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
ID,Name,Gender,Kor,Eng,Math
1,Albert,Male,80,60,80
2,Betty,Female,90,70,80
3,Charlie,Male,50,90,100
4,David,Male,60,90,90
5,Elsa,Female,80,80,90
```

- Reading and printing scores.csv in Python

예제 ex3-1.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data)
```

– (Line 1) Load pandas with the alias 'pd' – (Line 2)

pd.read_csv(filename): Command to load a CSV file – The loaded

variable 'data' has a data structure called a DataFrame • Similar to an Excel worksheet

Machine Translated by Google

Data extraction

- Extract rows that satisfy specific conditions from a DataFrame

문법 **데이터프레임에서 특정 조건을 만족하는 행을 추출하기 위한 명령어**

(데이터프레임 이름).loc[(행 조건문)]

- Extract only those with a Korean score of 80 or higher from the data

```
1  import pandas as pd
2  data = pd.read_csv('scores.csv')
3  print(data.loc[ data['Kor']>=80 ])
```

– (Line 3) Basically, it takes the form of data.loc[(condition)], and the structure is such that the condition corresponding to (condition) is 'data['Kor']>=80'.

Machine Translated by Google

Data Extraction (Continued)

- Extract specific columns from rows that satisfy specific conditions

문법 loc을 사용한 특정 행, 열 선택

(데이터프레임 이름).loc[(행 조건문), (열 이름)]

- If you want to select only the names of people with a Korean score of 80 or higher

```
print(data.loc[ data['Kor']>=80, 'Name'])
```

- Add 1 bonus point to the Korean scores of students who score 80 or higher.

```
import pandas as pd
data = pd.read_csv('scores.csv')
print(data)
data.loc[ data['Kor']>=80, 'Kor']+=1
print('<보너스 점수 반영>')
print(data)
```

Machine Translated by Google

Data grouping

- Aggregating data by grouping it according to specific criteria

문법 groupby를 이용한 데이터 집계

```
(데이터프레임 이름).groupby(열이름)
```

- Viewing the contents of each group: Use a for loop

문법 groups 안에서 각 그룹 확인

```
for (변수1),(변수2) in (그룹 변수 이름)
```

– (Variable 1): Group classification

– (Variable 2): Data for the corresponding group

Machine Translated by Google

Data Grouping (continued)

- Group and output data by gender

예제 ex3-2.py

```
import pandas as pd
data = pd.read_csv('scores.csv')
groups = data.groupby('Gender')
for g, d in groups:
    print('성별:', g)
    print(d)
    print('-----------------------------------')
```

Machine Translated by Google

Processing each row of a dataframe

• Number of rows in a DataFrame: len(DataFrameVariable) •

Retrieve a specific row number: DataFrameVariable.iloc[rowNumber] •

Calculate the average of Korean, English, and Math scores for each row

예제 ex3-3.py

```
import pandas as pd
data = pd.read_csv('scores.csv')
for i in range(len(data)):
    row=data.iloc[i]
    avg=(row['Kor']+row['Eng']+row['Math'])/3
    print(avg)
```

Machine Translated by Google

Processing each row of the dataframe (continued)

- Add the average value calculated from each row to the dataframe

예제 ex3-4.py

```
import pandas as pd
data = pd.read_csv('scores.csv')
print(data)
print('-----------------------------------------------')
avgList = []
for i in range(len(data)):
    row=data.iloc[i]
    avg=(row['Kor']+row['Eng']+row['Math'])/3
    avgList.append(avg)
print(avgList)
print('-----------------------------------------------')
data['Average']=avgList
print(data)
```

Machine Translated by Google

Saving DataFrames

- Save the dataframe with the added average column to a new Excel file

예제 ex3-5.py

```
import pandas as pd
data = pd.read_csv('scores.csv')
print(data)
print('-----------------------------------------------')
avgList = []
for i in range(len(data)):
    row=data.iloc[i]
    avg=(row['Kor']+row['Eng']+row['Math'])/3
    avgList.append(avg)
print(avgList)
print('-----------------------------------------------')
data['Average']=avgList
print(data)
data.to_csv('score2.csv')
```

Machine Translated by Google

matrix

- One or more numbers or characters arranged in a rectangular shape

- Advantages of matrix representation

  – Express complex expressions concisely

$$\begin{aligned} x + 2y &= 5 \\ 2x + y &= 4 \end{aligned} \Rightarrow \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix} \Rightarrow \mathrm{AX} = \mathrm{B}$$

- (Here, $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$를 A로, $\begin{bmatrix} x \\ y \end{bmatrix}$를 X로, $\begin{bmatrix} 5 \\ 4 \end{bmatrix}$ (Indicated)

  – Calculate the solution to the equation using the inverse matrix

- $\mathrm{AX} = \mathrm{B}$ Multiply both sides by the inverse of A

$$\mathrm{A}^{-1}\mathrm{AX} = \mathrm{A}^{-1}\mathrm{B}$$
$$\mathrm{X} = \mathrm{A}^{-1}\mathrm{B}$$

- The inverse of a matrix is $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ $\begin{bmatrix} ÿ0.33 & 0.67 \\ 0.67 & ÿ0.33 \end{bmatrix}$ since,

$$\mathrm{X} = \begin{bmatrix} ÿ0.33 & 0.67 \\ 0.67 & ÿ0.33 \end{bmatrix}\begin{bmatrix} 5 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Machine Translated by Google

NumPy package

- Provides the ability to create matrices and perform operations such as modifying values or performing calculations.

- Install NumPy –

  Open the command prompt and run pip install numpy (MacOS: pip3 install numpy)

- Importing and using the numpy package in a Python program
  – import numpy as np : Imports the numpy package with the name np

Machine Translated by Google

Lists and NumPy arrays

- Create a list and convert it to a NumPy array

예제 ex3-6.py

```
1 import numpy as np
2 a1 = [2,3,5]
3 print(a1)
4 print(type(a1))
5 a2 = np.array(a1)
6 print(a2)
7 print(type(a2))
8 print(a2.shape)
```

– (Lines 4, 7) type(variable name): Indicates the data type of the variable –

(Line 8) a2.shape: Indicates the shape of the variable (a2), i.e., the number of rows and columns

```
[2, 3, 5]
<class 'list'>
[2 3 5]
<class 'numpy.ndarray'>
(3,)
```

Machine Translated by Google

Lists and NumPy Arrays (continued)

- Modifying a specific value in

  an array – Modifying the first value of array a2 to 7: Similar to lists, a2[0]=7

```
import numpy as np
a1 = [2,3,5]
a2 = np.array(a1)
print(a2)
a2[0]=7
print(a2)
```

- Advantages of NumPy arrays

  – Various NumPy features can be applied –

  Processing speed is much faster than lists and occupies less memory • NumPy

  arrays are optimized for numerical operations on large amounts of data

Machine Translated by Google

Element-wise operations on NumPy arrays

- Addition and subtraction of each element of a NumPy array

```
import numpy as np
c = np.array([1,2,3])
d = np.array([2,3,5])

print(c)
print(d)
print(c+d)
print(c-d)
```

```
[1 2 3]
[2 3 5]
[3 5 8]
[-1 -1 -2]
```

Machine Translated by Google

Element-wise operations on NumPy arrays (continued)

- Element-wise multiplication and division of NumPy arrays

```
import numpy as np
c = np.array([1,2,3])
d = np.array([2,3,5])

print(c*d)
print(c/d)
```

```
[2 6 15]
[0.5 0.66666667 0.6]
```

Machine Translated by Google

Matrix operations on NumPy arrays

- Multiplication of two matrices: Each row of the left matrix is multiplied by each column of the right matrix
  - 만약 $A = [a, b, c]$ 이고 $B = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ 라면, $AB = ax + by + cz$ 가 되는 방식

- In NumPy, use the dot() function for matrix multiplication.

```
import numpy as np
c = np.array([1,2,3])
d = np.array([2,3,5])

print(np.dot(c,d))
print(c.dot(d))
```

Machine Translated by Google

Full-scale matrix operations

- Definition of a 2D Array (Matrix)

```
import numpy as np
a = np.array([[2,4,6], [3,5,7]])
print(a)
print('행렬 형태: ', a.shape)
```

- Output elements of a specific row or column

```
print(a[0,0])
print(a[0,2])
print(a[1,1])
```

Machine Translated by Google

Element-wise arithmetic operations on a matrix

- Apply basic arithmetic operations to matrices element by element, just like with one-dimensional arrays

예제 ex3-8.py

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
b = np.array([[2,4,6],[3,5,7]])
print(a+b)
print(a-b)
print(a*b)
print(a/b)
```

```
[[ 3  6  9]
 [ 7 10 13]]
[[-1 -2 -3]
 [ 1  0 -1]]
[[ 2  8 18]
 [12 25 42]]
[[0.5        0.5        0.5        ]
 [1.33333333 1.         0.85714286]]
```

Machine Translated by Google

Matrix multiplication operation

- Matrix multiplication is expressed using the dot() function in NumPy
  - The number of columns on the left and the number of rows on the right must match
  - The following program will cause an error • a

    has 3 columns, but b has 2 rows

```
1  import numpy as np
2  a = np.array([[1,2,3],[4,5,6]])
3  b = np.array([[2,4,6],[3,5,7]])
4  print(np.dot(a,b))
```

```
Traceback (most recent call last):
  File "D:/Dropbox/2.Lecture/BusinessAnalytics_2021년_2학기/202108_BA_book/draft_v0.2/
BA_code/ch03_Pandas,numpy/ex3-7.py", line 4, in <module>
    print(np.dot(a,b))
  File "<__array_function__ internals>", line 6, in dot
ValueError: shapes (2,3) and (2,3) not aligned: 3 (dim 1) != 2 (dim 0)
```

Machine Translated by Google

Matrix Multiplication Operation (continued)

Transpose b to transform it into a 3x2 matrix and recalculate

– Use the np.transpose() function for the transpose matrix.

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
b = np.array([[2,4,6],[3,5,7]])
b = np.transpose(b)
print(a)
print(b)
print(np.dot(a,b))
```

```
[[1 2 3]
 [4 5 6]]
[[2 3]
 [4 5]
 [6 7]]
[[28 34]
 [64 79]]
```

Machine Translated by Google

identity matrix

- A square matrix where only the diagonal elements are 1 and all others are 0
  - The property that multiplying the identity matrix by any other matrix does not change the original matrix
  - Created using np.eye(number of rows)

```
import numpy as np
id = np.eye(3)
print(id)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Machine Translated by Google

inverse matrix

- A matrix that becomes the identity matrix when multiplied by a specific matrix – Use np.linalg.inv()

```
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.linalg.inv(a)
print(a)
print(b)
print(np.dot(a,b))
```

– A slight decimal error may occur

- $8.8817842e-16 = 8.8817842 * 10^{-16} = 0.00000000000000088817842$ (Almost 0)

```
[[1 2]
 [3 4]]
[[-2.   1. ]
 [ 1.5 -0.5]]
[[1.0000000e+00 0.0000000e+00]
 [8.8817842e-16 1.0000000e+00]]
```

Machine Translated by Google

Generation of various special matrices

- Zero Matrix: A matrix where all components are 0 –

  np.zeros(number of rows, number of

  columns) – Frequently used for matrix initialization

```
import numpy as np
zero = np.zeros((3,5))
print(zero)
```

- Other Special Matrices –

  np.ones(number of rows, number of columns): A matrix where all elements are 1 –

  np.full((number of rows, number of columns), initialization value): A matrix where all elements are the initialization value

Machine Translated by Google

Extraction of statistical values from data

- Mean, standard deviation, median, etc.
  - calculated using np.mean(), np.stdev(), and np.median(), respectively

예제 ex3-9.py

```
import numpy as np
a = np.array([1,2,3,5,9])
print('평균: ', np.mean(a))
print('표준편차: ', np.std(a))
print('중앙값: ', np.median(a))
print('최소값: ', np.min(a))
print('최대값: ', np.max(a))
```